

PRACTICAL_WEEK-11

Task 1: Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
printf("Starting multithreaded counter...\n");
printf("Number of Threads: %d\n", NUM_THREADS);
printf("Increments per Thread: %d\n\n", INCREMENTS);

for (int i = 0; i < NUM_THREADS; i++)
{
    if (pthread_create(&threads[i], NULL,
                     increment_counter, NULL) != 0)
    {
        printf("Error creating thread %d\n", i + 1);
        return 1;
    }
}

for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}

long expected = NUM_THREADS * INCREMENTS;

printf("Expected Counter Value : %ld\n", expected);
printf("Actual Counter Value   : %ld\n", counter);

if (counter != expected)
    printf("Race condition observed!\n");
else
    printf("No visible race condition in this run.\n");

return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/practical11_task1.c -o bin/practical11_task1 -pthread
./bin/practical11_task1
Starting multithreaded counter...
Number of Threads: 4
Increments per Thread: 100000

Expected Counter Value : 400000
Actual Counter Value   : 103575
Race condition observed!
amrutha@Amruthas-MacBook-Air OSSP % ./bin/practical11_task1
./bin/practical11_task1
Starting multithreaded counter...
Number of Threads: 4
Increments per Thread: 100000

Expected Counter Value : 400000
Actual Counter Value   : 104620
Race condition observed!
Starting multithreaded counter...
Number of Threads: 4
Increments per Thread: 100000

Expected Counter Value : 400000
Actual Counter Value   : 107275
Race condition observed!
Starting multithreaded counter...
Number of Threads: 4
Increments per Thread: 100000

Expected Counter Value : 400000
Actual Counter Value   : 101031
Race condition observed!
amrutha@Amruthas-MacBook-Air OSSP %
```

Task 2: Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```
pthread_mutex_init(&lock, NULL);

printf("Starting synchronized counter...\n");
printf("Number of Threads: %d\n", NUM_THREADS);
printf("Increments per Thread: %d\n\n", INCREMENTS);

for (int i = 0; i < NUM_THREADS; i++)
{
    if (pthread_create(&threads[i], NULL,
                      increment_counter, NULL) != 0)
    {
        printf("Error creating thread %d\n", i + 1);
        return 1;
    }
}

for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}

long expected = NUM_THREADS * INCREMENTS;

printf("Expected Counter Value : %ld\n", expected);
printf("Actual Counter Value   : %ld\n", counter);

if (counter == expected)
    printf("Synchronization successful!\n");
else
    printf("Unexpected counter mismatch.\n");

pthread_mutex_destroy(&lock);

return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % nano src/practical11_task2.c
amrutha@Amruthas-MacBook-Air OSSP % gcc src/practical11_task2.c -o bin/practical11_task2 -pthread
amrutha@Amruthas-MacBook-Air OSSP % ./bin/practical11_task2
Starting synchronized counter...
Number of Threads: 4
Increments per Thread: 100000

Expected Counter Value : 400000
Actual Counter Value   : 400000
Synchronization successful!
amrutha@Amruthas-MacBook-Air OSSP %
```